SQL Server System Tables and Views

By Don Schlichting

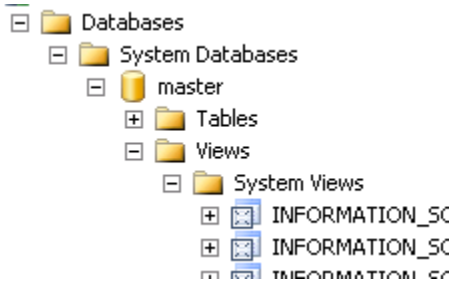
This article will explore various options for obtaining SQL Server metadata information.

Introduction

When a SQL Server object is created, its properties are called metadata. The metadata is stored in special System Tables. For example, in SQL 2000, when a new column was created, the column name and data type could be found in an internal System Table called syscolumns. All SQL objects produce metadata. Every time SQL 2000 Enterprise Manager or SQL 2005 SQL Server Management Studio is browsed, the information displayed about database, tables, and all objects, comes from this metadata. There are many uses for this metadata, including gathering performance statistics, discovering table and column similarities and differences during a database upgrade, and obtaining lock information. In previous versions of SQL Server, these System Tables were exposed and could be queried like any standard table. However, starting with SQL 2005, System Tables are hidden and they cannot be directly queried. Even with full DBA rights, System Tables are restricted. Although not directly accessible, there are built in views and procedures for extracting metadata. Some of these are new in SQL 2005; others were carried forward from pervious versions. Most have the advantage of being more readable and self-describing than querying System Tables. If you have legacy scripts directly referencing System Tables, there are many new System Views that will directly take their place.

System Views

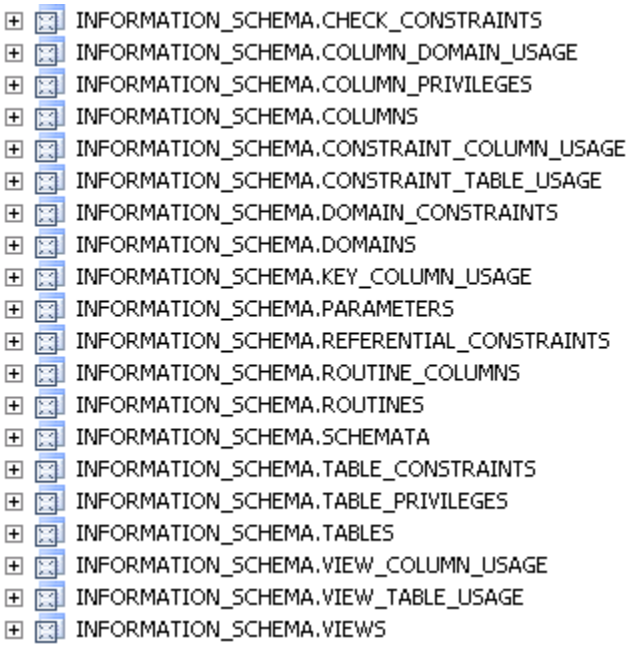
System Views are predefined Microsoft created views for extracting SQL Server metadata. There are over 230 various System Views. To display all the views in SQL Server, launch the SQL Management Studio; expand Databases, System Databases, and select master, Views, System Views.



These System Views will be automatically inserted into any user created database. The System Views are grouped into several different schemas. In SQL 2005, schemas are used as security containers. There can be several different schemas inside a single database. This is a better ANSI implementation of schemas compared to their use in SQL 2000. See Marcin Policht's excellent article; SQL Server 2005 Security, at http://www.databasejournal.com/features/mssql/article.php/3481751 for a detailed explanation of SQL 2005 schemas and security.

Information Schema

The first group of System Views belongs to the Information Schema set. Information Schema is an ANSI specification for obtaining metadata. There are twenty different views for displaying most physical aspects of a database, such as table, column, and view information.



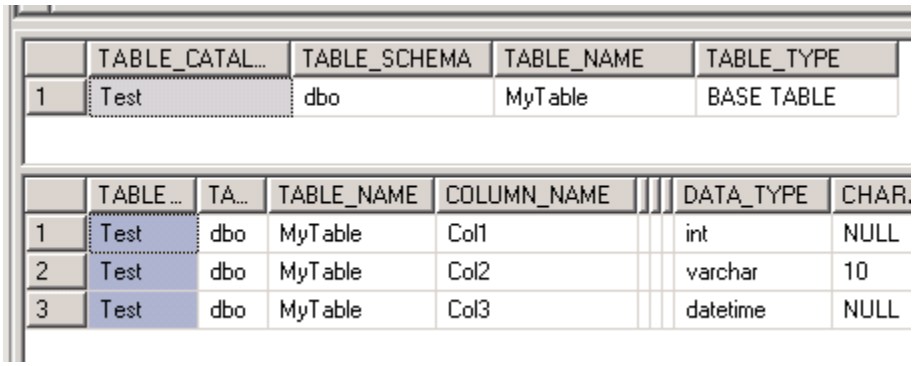
Information Schema views were available in SQL Server and should continue to appear in future versions of SQL. They are a few ANSI terms that translate differently in SQL. An ANSI "Catalog" is a SQL "Database"; an ANSI "RowVersion" is a SQL "Timestamp"; and an ANSI "Timestamp" is a SQL "DateTime." Aside from this, Information Schema views are easy to implement. For an example, we will create a small table with a few columns.

```
CREATE DATABASE Test
GO
USE Test
GO
CREATE TABLE MyTable
(
Col1 int,
Col2 varchar(10),
Col3 datetime
)
GO
```

To use Information Schema views, select them like any standard view. The following TSQL will display column and table information on the new database;

```
SELECT *
FROM INFORMATION_SCHEMA.TABLES
```

```
SELECT *
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'MyTable'
```



	TABLE_CATAL...	TABLE_SCHEMA	TABLE_NAME	TABLE_TYPE
1	Test	dbo	MyTable	BASE TABLE

	TABLE...	TA...	TABLE_NAME	COLUMN_NAME	DATA_TYPE	CHAR...
1	Test	dbo	MyTable	Col1	int	NULL
2	Test	dbo	MyTable	Col2	varchar	10
3	Test	dbo	MyTable	Col3	datetime	NULL

Most of the Information Schema view names are self-explanatory. INFORMATION_SCHEMA.TABLES returns a row for each table. INFORMATION_SCHEMA.COLUMNS returns a row for each column. A few though, refer to ANSI names. INFORMATION_SCHEMA.COLUMN_DOMAIN_USAGE contains a row for each column created with a user-defined type, and INFORMATION_SCHEMA.DOMAIN lists a row for each user-defined type. INFORMATION_SCHEMA.ROUTINES shows a record for each stored procedure or function. A benefit to Information Schema views is that because they are an ANSI standard, you will find them in many other database packages.

Catalog Views

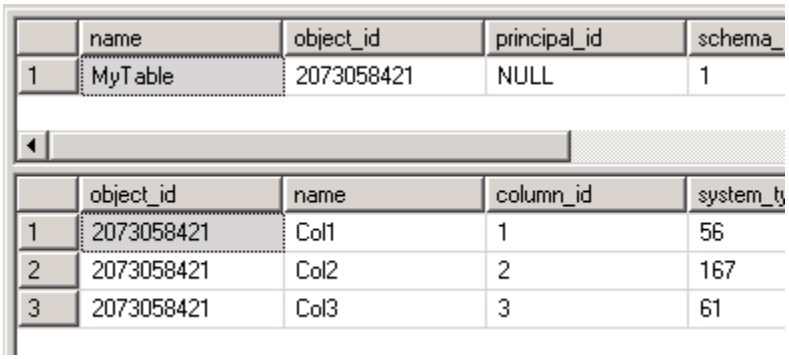
New in SQL 2005 are Catalog Views. Microsoft recommends them as the most general interface to the catalog metadata. They are efficient and all user available catalog metadata is exposed. The amount of views is impressive. Best of all, many of the columns returned by Catalog Views are self-describing. Documentation organizes Catalog Views into several different groups:

- Partition Function Catalog Views
- Server-wide Configuration Catalog Views
- Data Spaces and Fulltext Catalog Views
- Databases and Files Catalog Views
- CLR Assembly Catalog Views
- Schemas Catalog View
- Scalar Types Catalog Views
- Security Catalog Views
- Objects Catalog Views
- Database Mirroring Catalog Views
- Messages (For Errors) Catalog Views
- XML Schemas (XML Type System) Catalog Views
- Service Broker Catalog Views
- Linked Servers Catalog Views
- HTTP Endpoints Catalog Views
- Extended Properties Catalog Views

The views we need to gather table and column information, like the previous example, are grouped under "Objects Catalog Views". This group includes views on tables, columns, indexes, constraints, and triggers to name a few. Our example requires two views, "sys.tables" and "sys.columns." The columns view will need to be joined on the table view as shown below.

```
SELECT *
FROM sys.tables

SELECT *
FROM sys.columns INNER JOIN sys.tables ON
          sys.tables.object_id = sys.columns.object_id
WHERE sys.tables.name = 'MyTable'
```



	name	object_id	principal_id	schema_
1	MyTable	2073058421	NULL	1

	object_id	name	column_id	system_ty
1	2073058421	Col1	1	56
2	2073058421	Col2	2	167
3	2073058421	Col3	3	61

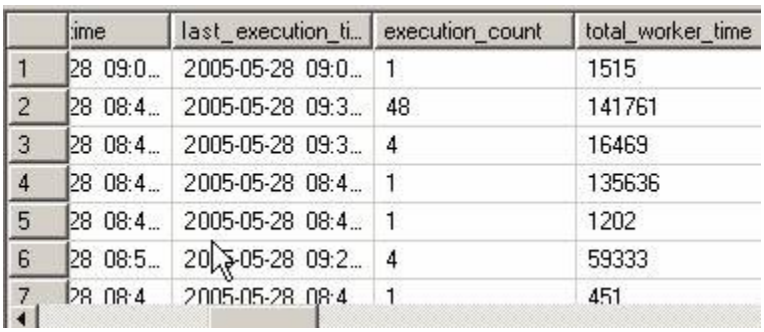
Sys All

There are four views in a Sys_All group. These views contain information about the System Views as well as user created objects. The views are sys.all_columns, sys.all_objects, sys.all_parameters, and sys.all_views.

Dynamic Management Views

The last groups of views are called Dynamic Management views, or DM. They are used to gather statistics stored in memory but not persistent on disk such as thread information, memory usage, and connection details. These offer administrators a fast and reliable method for obtaining performance numbers. For example, to show the statistics for cached queries, execute this DM statement:

```
SELECT *
FROM sys.dm_exec_query_stats
```

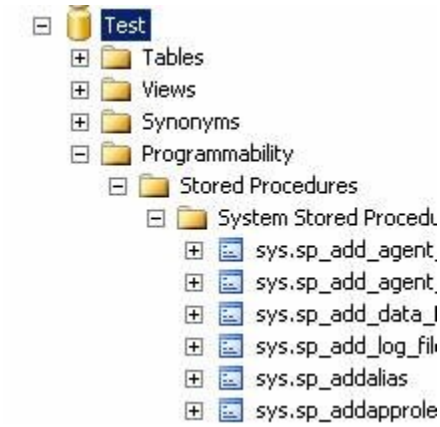


	ime	last_execution_ti...	execution_count	total_worker_time
1	28 09:0...	2005-05-28 09:0...	1	1515
2	28 08:4...	2005-05-28 09:3...	48	141761
3	28 08:4...	2005-05-28 09:3...	4	16469
4	28 08:4...	2005-05-28 08:4...	1	135636
5	28 08:4...	2005-05-28 08:4...	1	1202
6	28 08:5...	20[illegible]-05-28 09:2...	4	59333
7	28 08:4	2005-05-28 08:4	1	451

These DM views will become invaluable for many DBAs.

System Stored Procedures

In addition to the System Views, there are many System Stored Procedures that can be used for administrative purposes. These pre-made procedures return results similar to System Views. They are located under each database, Programmability, Stored Procedures, and System Stored Procedures. They belong to sys schema.

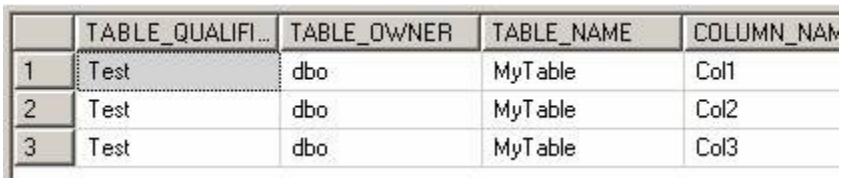


To obtain column information using a System Stored Procedure, execute sp_columns with the following script:

EXEC sys.sp_columns 'MyTable'

Conclusion

For obtaining SQL Sever metadata information, SQL Server offers a large variety of pre-made views and procedures. They are easy and fast to implement and usually return information that is far less cryptic than the tools provided in previous versions.



	TABLE_QUALIFI...	TABLE_OWNER	TABLE_NAME	COLUMN_NAM
1	Test	dbo	MyTable	Col1
2	Test	dbo	MyTable	Col2
3	Test	dbo	MyTable	Col3

Conclusion

For obtaining SQL Sever metadata information, SQL Server offers a large variety of pre-made views and procedures. They are easy and fast to implement and usually return information that is far less cryptic than the tools provided in previous versions.